AI MINI PROJECT REPORT

ON

ROAD ACCIDENT SEVERITY PREDICTOR

Using Machine Learning and Random Forest Classification

Submitted By

Abhinav Ghogare

B.Tech Information Technology

Priyadarshini College of Engineering, Nagpur

Academic Year: 2026–27

1. Aim

To design and implement an AI-based Road Accident Severity Predictor that estimates whether a road accident is Minor, Moderate, or Severe from accident and environmental conditions using a machine learning classification model.

2. Problem Definition

Road accidents can vary greatly in severity depending on speed, weather, road type, visibility, traffic conditions, road surface, number of vehicles, and alcohol involvement. Manually assessing the likely severity of an accident from several factors is difficult. The proposed system uses historical accident records and a trained machine learning model to predict severity for a new accident case.

3. Abstract

The Road Accident Severity Predictor is a simple AI/ML mini project developed using Python, Pandas, Scikit-learn, and Tkinter. The system uses accident-related features such as weather, road type, lighting, road surface, speed, vehicles involved, visibility, traffic density, and alcohol involvement. Categorical values are encoded using One-Hot Encoding and a Random Forest Classifier is trained on the dataset. The trained model predicts one of three severity classes: Minor, Moderate, or Severe. A graphical interface allows the user to enter accident conditions and view the predicted severity together with prediction confidence.

4. Objectives

• To develop a simple AI-based accident severity prediction system.
• To prepare and use a structured road accident dataset.
• To preprocess categorical and numerical accident features.
• To train a Random Forest classification model.
• To predict Minor, Moderate, or Severe accident severity.
• To provide the prediction through a simple graphical interface.

5. Introduction

Accident severity prediction is a classification problem in which an intelligent system learns relationships between accident conditions and their observed severity. Machine learning can assist by identifying patterns in historical data and applying those patterns to new cases. This project demonstrates the complete mini-project workflow: dataset preparation, preprocessing, model training, evaluation, and prediction through a GUI.

6. Technologies Used

• Programming Language: Python

• GUI Framework: Tkinter

• Data Processing: Pandas and NumPy

• Machine Learning: Scikit-learn

• AI Technique: Supervised Machine Learning

• Classification Algorithm: Random Forest

• Development Environment: Visual Studio Code

7. AI / Machine Learning Technique

7.1 Supervised Classification

The model learns from labelled accident records where each row contains input features and a known severity class. After training, the model predicts the class of an unseen accident record.

7.2 Random Forest Classifier

Random Forest combines multiple decision trees and uses their collective prediction to classify an input. It is suitable for this mini-project because it can handle mixed feature types after preprocessing and can model non-linear relationships between accident conditions and severity.

7.3 One-Hot Encoding

Categorical fields such as weather, road type, and traffic density are converted into numerical indicator features using One-Hot Encoding before being passed to the classifier.

8. Dataset and Features

The included project dataset contains 1,200 accident records. Each record has nine input features and one target feature named Severity. The dataset is included in the ZIP so the project can be executed directly without obtaining another file.

Feature	Type	Description
Weather	Categorical	Clear, Rain, Fog, Storm
Road_Type	Categorical	Highway, City, Rural, Intersection
Light_Condition	Categorical	Daylight, Night, Dawn/Dusk
Road_Surface	Categorical	Dry, Wet, Icy
Speed_kmph	Numeric	Vehicle speed in km/h
Vehicles_Involved	Numeric	Number of vehicles involved
Visibility_km	Numeric	Approximate visibility in kilometres
Traffic_Density	Categorical	Low, Medium, High
Alcohol_Involvement	Categorical	Yes or No
Severity	Target	Minor, Moderate, Severe

9. System Requirements

Hardware:

• Computer or laptop

• Minimum 4 GB RAM recommended

Software:

• Python 3.x

• Visual Studio Code or another Python IDE

• Required packages listed in requirements.txt

10. Input Parameters

• Weather

• Road type

• Light condition

• Road surface

• Speed in km/h

• Number of vehicles involved

• Visibility in km

• Traffic density

• Alcohol involvement

11. Target Classes

• Minor – lower predicted accident severity.

• Moderate – intermediate predicted accident severity.

• Severe – higher predicted accident severity.

12. Working / Methodology

Step 1: Load the road accident dataset.
Step 2: Separate input features and the Severity target.
Step 3: Divide categorical and numerical features.
Step 4: Apply One-Hot Encoding to categorical features.
Step 5: Split the data into training and testing sets.
Step 6: Train a Random Forest Classifier.
Step 7: Evaluate the trained model on test data.
Step 8: Accept new accident conditions from the GUI.
Step 9: Pass the input through the same preprocessing and trained model.
Step 10: Display the predicted severity and class probabilities.

13. Algorithm

1. Start the application.
2. Read the accident dataset.
3. Separate features and target.
4. Encode categorical features.
5. Split data into training and testing sets.
6. Train the Random Forest classifier.
7. Read new accident details.
8. Apply the trained preprocessing pipeline.
9. Predict the severity class.
10. Calculate prediction probabilities.
11. Display the result.
12. Stop.

14. Program

```
import tkinter as tk
from tkinter import ttk, messagebox
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

DATA_FILE = "road_accident_data.csv"

df = pd.read_csv(DATA_FILE)
X = df.drop(columns=["Severity"])
y = df["Severity"]

categorical = [
    "Weather", "Road_Type", "Light_Condition", "Road_Surface",
    "Traffic_Density", "Alcohol_Involvement"
]
numeric = ["Speed_kmph", "Vehicles_Involved", "Visibility_km"]

preprocessor = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
    ("num", "passthrough", numeric)
])

model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(
        n_estimators=150, random_state=42, max_depth=10
    ))
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
model.fit(X_train, y_train)

def predict_severity():
    try:
        record = {
            "Weather": weather_box.get(),
            "Road_Type": road_box.get(),
            "Light_Condition": light_box.get(),
            "Road_Surface": surface_box.get(),
            "Speed_kmph": float(speed_entry.get()),
            "Vehicles_Involved": int(vehicle_entry.get()),
            "Visibility_km": float(visibility_entry.get()),
            "Traffic_Density": traffic_box.get(),
            "Alcohol_Involvement": alcohol_box.get()
        }

        if record["Speed_kmph"] <= 0 or record["Vehicles_Involved"] <= 0 or record["Visibility_km"] <= 0:
            raise ValueError

        input_df = pd.DataFrame([record])
        prediction = model.predict(input_df)[0]
        probabilities = model.predict_proba(input_df)[0]
        classes = model.named_steps["classifier"].classes_

        result.delete("1.0", tk.END)
        result.insert(tk.END, "ROAD ACCIDENT SEVERITY PREDICTION\n")
        result.insert(tk.END, "=" * 42 + "\n\n")
        result.insert(tk.END, f"Predicted Severity: {prediction}\n\n")
        result.insert(tk.END, "Prediction Confidence:\n")
```

```
        for label, probability in zip(classes, probabilities):
            result.insert(tk.END, f"{label}: {probability * 100:.2f}%\n")

    except (ValueError, TypeError):
        messagebox.showerror(
            "Input Error",
            "Enter valid numeric values for speed, vehicles and visibility."
        )

root = tk.Tk()
root.title("Road Accident Severity Predictor")
root.geometry("850x700")
root.resizable(False, False)

tk.Label(
    root, text="ROAD ACCIDENT SEVERITY PREDICTOR",
    font=("Arial", 20, "bold")
).pack(pady=15)

frame = tk.Frame(root)
frame.pack(pady=5)

def add_row(row, label, widget):
    tk.Label(frame, text=label, width=24, anchor="w").grid(
        row=row, column=0, padx=8, pady=7
    )
    widget.grid(row=row, column=1, padx=8, pady=7)

weather_box = ttk.Combobox(frame, values=["Clear","Rain","Fog","Storm"], state="readonly", width=22)
weather_box.set("Clear")
add_row(0, "Weather:", weather_box)

road_box = ttk.Combobox(frame, values=["Highway","City","Rural","Intersection"], state="readonly", width=22)
road_box.set("City")
add_row(1, "Road Type:", road_box)

light_box = ttk.Combobox(frame, values=["Daylight","Night","Dawn/Dusk"], state="readonly", width=22)
light_box.set("Daylight")
add_row(2, "Light Condition:", light_box)

surface_box = ttk.Combobox(frame, values=["Dry","Wet","Icy"], state="readonly", width=22)
surface_box.set("Dry")
add_row(3, "Road Surface:", surface_box)

speed_entry = tk.Entry(frame, width=25)
speed_entry.insert(0, "50")
add_row(4, "Speed (km/h):", speed_entry)

vehicle_entry = tk.Entry(frame, width=25)
vehicle_entry.insert(0, "2")
add_row(5, "Vehicles Involved:", vehicle_entry)

visibility_entry = tk.Entry(frame, width=25)
visibility_entry.insert(0, "8")
add_row(6, "Visibility (km):", visibility_entry)

traffic_box = ttk.Combobox(frame, values=["Low","Medium","High"], state="readonly", width=22)
traffic_box.set("Medium")
add_row(7, "Traffic Density:", traffic_box)

alcohol_box = ttk.Combobox(frame, values=["No","Yes"], state="readonly", width=22)
alcohol_box.set("No")
add_row(8, "Alcohol Involvement:", alcohol_box)

tk.Button(
    root, text="Predict Accident Severity",
    command=predict_severity, width=28
```

```
).pack(pady=15)

result = tk.Text(root, width=80, height=12, font=("Arial", 10))
result.pack(pady=10)

root.mainloop()
```

15. Sample Output

The included model achieved an accuracy of **68.33%** on the held-out test set for the included dataset.

Sample input: Rain, Highway, Night, Wet, 85 km/h, 3 vehicles, 3.0 km visibility, High traffic, Alcohol = Yes.

Predicted Severity: Moderate

Prediction probabilities: Minor: 18.2%, Moderate: 42.2%, Severe: 39.6%

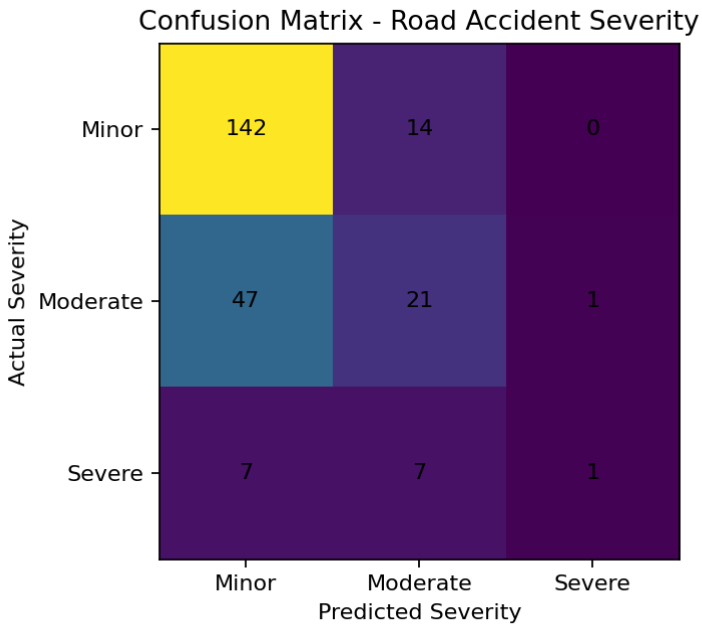


Figure 1: Confusion matrix generated during model evaluation.

16. Advantages

• Simple and easy-to-use graphical interface.

• Uses an actual machine learning classification model.

• Combines categorical and numerical accident features.

• Provides three interpretable severity classes.

• Shows prediction confidence through class probabilities.

• Complete dataset and source code are included with the project.

17. Limitations

• The included dataset is a prepared mini-project dataset and is not a replacement for an official national accident database.

• Predictions depend on the quality and representativeness of the training data.

• The model does not account for every real-world factor such as vehicle safety systems, exact collision geometry, emergency response time, or road design details.

• The current application is intended for academic demonstration and not for real-world emergency or legal decisions.

18. Future Scope

The system can be extended using larger official datasets, real-time weather and traffic information, GPS/location features, explainable AI methods, model comparison, a web/mobile interface, and periodic retraining. These features are outside the scope of the current mini-project.

19. Conclusion

The Road Accident Severity Predictor demonstrates how supervised machine learning can be applied to a practical road-safety problem. By preprocessing accident features and training a Random Forest classifier, the system predicts Minor, Moderate, or Severe severity for a new accident case. The project provides a complete and understandable academic example covering dataset preparation, machine learning, evaluation, and GUI-based prediction.

20. Screenshots / Output

Run the application using the command shown below:

```
pip install -r requirements.txt
python road_accident_severity_predictor.py
```

The application opens a form where the user enters accident conditions and presses “Predict Accident Severity”. The result area displays the predicted class and probabilities.

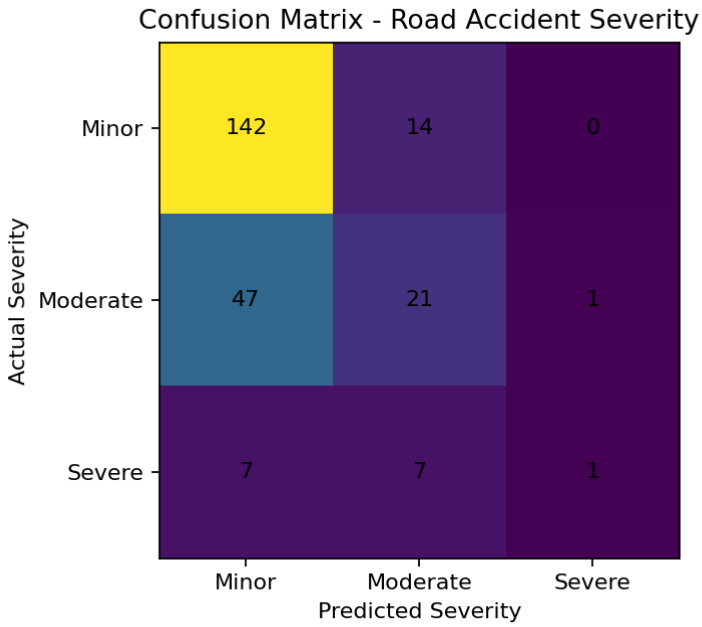


Figure 2: Model evaluation output generated from the included dataset.

Project files included in the ZIP:

road_accident_severity_predictor.py, road_accident_data.csv, requirements.txt, sample_output.txt, confusion_matrix.png, README.txt, and this project report.